



مهلت: ۲۶ آذر

شبکه‌های کامپیوتری
Fall 2025



تمرین چهارم

سؤال ۱: مسیریابی با طولانی‌ترین تطابق پیشنهاد

یک روتر را در نظر بگیرید که در یک شبکه دیتاگرامی از آدرس‌های ۸ بیتی استفاده می‌کند. این روتر از مکانیزم Longest Prefix Matching برای مسیریابی بسته‌ها استفاده می‌کند. جدول ارسال این روتر مطابق جدول زیر است:

Interface	Prefix Match
۱	۱۰
۲	۱۱
۳	۱۰۰
۴	۱۱۱
۵	...
۶	otherwise

۱.۱. به پرسش‌های زیر به صورت تشریحی پاسخ دهید:

- (آ) توضیح دهید اگر دو ورودی جدول دقیقاً پیشنهادی هم‌طول داشته باشند که هر دو با یک آدرس مقصد تطابق داشته باشند، روتر بر اساس چه معیارهایی می‌تواند بین آن‌ها انتخاب کند. یک مثال ساده بزنید.
- (ب) اگر ورودی default route (سطر otherwise) از جدول حذف شود، برای بسته‌هایی که با هیچ پیشنهادی تطابق ندارند چه اتفاقی می‌افتد؟ از منظر عملی و پروتکل‌های لایه شبکه توضیح دهید.

۲.۱. برای هر یک از آدرس‌های زیر، تعیین کنید بسته به کدام Interface ارسال می‌شود. از قاعده طولانی‌ترین تطابق پیشنهاد استفاده کنید:

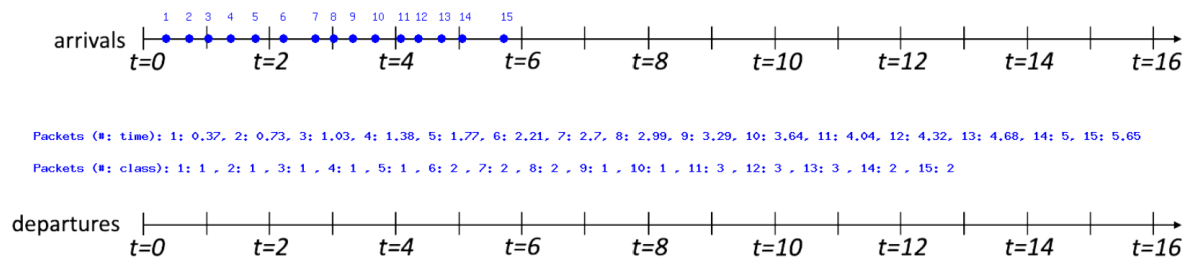
(آ) 00000011

(ب) 10101110

(ج) 10001100

سؤال ۲: زمان‌بندی بسته‌ها در روتر (Packet Scheduling)

شکل زیر زمان رسیدن ۱۵ بسته را به خروجی یک روتر در بازه زمانی $[0, 5]$ نشان می‌دهد. زمان را slotted در نظر می‌گیریم؛ به‌طوری‌که هر اسلات زمانی در لحظات صحیح $t = 0, 1, 2, \dots$ شروع می‌شود. بسته‌ها می‌توانند در هر لحظه‌ای از اسلات وارد شوند و در هر اسلات ممکن است چندین بسته وارد صف شوند. در ابتدای هر اسلات، زمان‌بند scheduler یکی از بسته‌های موجود در صف را برای ارسال انتخاب می‌کند. هر بسته دقیقاً یک اسلات زمانی برای ارسال نیاز دارد؛ بنابراین بسته‌ای که در زمان t انتخاب می‌شود، در پایان اسلات و در زمان $t+1$ ارسالش کامل می‌شود.



شکل ۱: زمان رسیدن بسته‌ها، کلاس هر بسته و اسلاتهای خروجی روتر

فرض کنید روتر از الگوریتم Weighted Fair Queueing (WFQ) برای زمان‌بندی استفاده می‌کند. سه کلاس ترافیک (1, 2, 3) داریم که عدد کوچک‌تر بیانگر اولویت بالاتر است و وزن کلاس‌ها در WFQ به‌ترتیب 0.3, 0.5 و 0.2 است.

۱.۲. به پرسش‌های زیر به‌صورت تشریحی پاسخ دهید:

(آ) ایده اصلی الگوریتم WFQ را توضیح دهید و بیان کنید این الگوریتم از چه نظر نسبت به Priority Queueing منصفانه‌تر عمل می‌کند.

(ب) آیا در WFQ با این وزن‌ها امکان starvation برای کلاس ۳ وجود دارد؟ استدلال کنید که چرا بله یا چرا خیر، و این وضعیت را با الگوریتم Priority Queueing مقایسه کنید.

۲.۲. با فرض استفاده از الگوریتم WFQ و بر اساس داده‌های شکل بالا، ترتیب ارسال بسته‌ها را از زمان $t=1$ تا $t=15$ مشخص کنید؛ یعنی در هر اسلات زمانی بنویسید کدام شماره بسته ارسال می‌شود.

سؤال ۳: Subnet Addressing

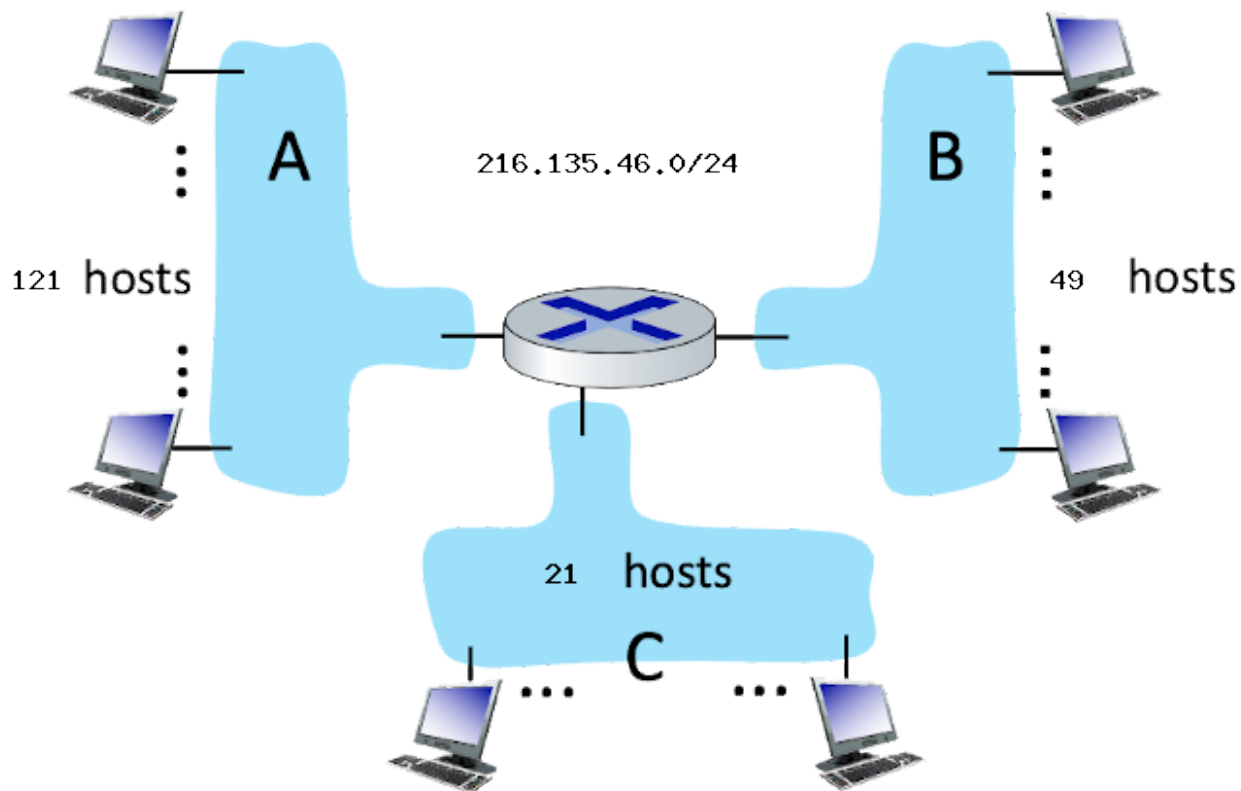
روتری را در نظر بگیرید که به سه زیرشبکه A، B و C متصل است. این سه زیرشبکه ۲۴ بیت بالایی آدرس را به صورت مشترک دارند و فضای آدرس آن‌ها به صورت $216.135.46.0/24$ داده شده است. تعداد میزبان‌های مورد نیاز در هر زیرشبکه به صورت زیر است:

- زیرشبکه A: 121 میزبان

- زیرشبکه B: 49 میزبان

- زیرشبکه C: 21 میزبان

آدرس‌های IP باید به گونه‌ای بین این سه زیرشبکه تقسیم شوند که برای هر زیرشبکه کمترین مقدار ممکن فضای آدرس اختصاص داده شود (بدون کمبود آدرس) و تا حد امکان یک فضای آدرس پیوسته بزرگ برای اضافه شدن یک زیرشبکه جدید در آینده آزاد بماند.



۱.۳. برای سوالات زیر توضیح مختصر بیاورید:

(آ) توضیح دهید آیا فضای آدرس $216.135.46.0/24$ عمومی است یا خصوصی و چرا.

(ب) نشان دهید در یک بلوک 24/ حداکثر چند میزبان می‌توان داشت و علت کم کردن ۲ آدرس از کل فضای آدرس را به صورت مفهومی توضیح دهید.

۲.۳. طراحی زیرشبکه‌ها

(آ) با توجه به تعداد میزبان‌های مورد نیاز در هر زیرشبکه، آدرس شبکه (به صورت CIDR) برای هر یک از زیرشبکه‌های A، B و C را به گونه‌ای تعیین کنید که شروط گفته شده در صورت سؤال برقرار باشد. (پاسخ خود را به صورت $A: 216.135.46.x/yy$ و غیره بنویسید.)

(ب) برای زیرشبکه A که در بخش قبل طراحی کردید، آدرس broadcast زیرشبکه A، اولین آدرس قابل تخصیص به میزبان و آخرین آدرس قابل تخصیص به میزبان در زیرشبکه A را مشخص کنید.

(ج) برای زیرشبکه B که در بخش قبل طراحی کردید، آدرس broadcast زیرشبکه B، اولین آدرس قابل تخصیص به میزبان و آخرین آدرس قابل تخصیص به میزبان در زیرشبکه B را مشخص کنید.

(د) برای زیرشبکه C که در بخش قبل طراحی کردید، آدرس broadcast زیرشبکه C، اولین آدرس قابل تخصیص به میزبان و آخرین آدرس قابل تخصیص به میزبان در زیرشبکه C را مشخص کنید.

سوال ۴: Network Address Translation

در شکل زیر سه میزبان با آدرس‌های خصوصی 10.0.1.14، 10.0.1.15 و 10.0.1.18 در پشت یک روتر NAT قرار دارند. روتر دارای آدرس 10.0.1.26 در سمت LAN و آدرس 135.122.203.205 در سمت WAN است. فرض کنید میزبان 10.0.1.14 یک دیتاگرام به میزبان 128.119.171.185 ارسال می‌کند. پورت مبدا 3364 و پورت مقصد 80 است.

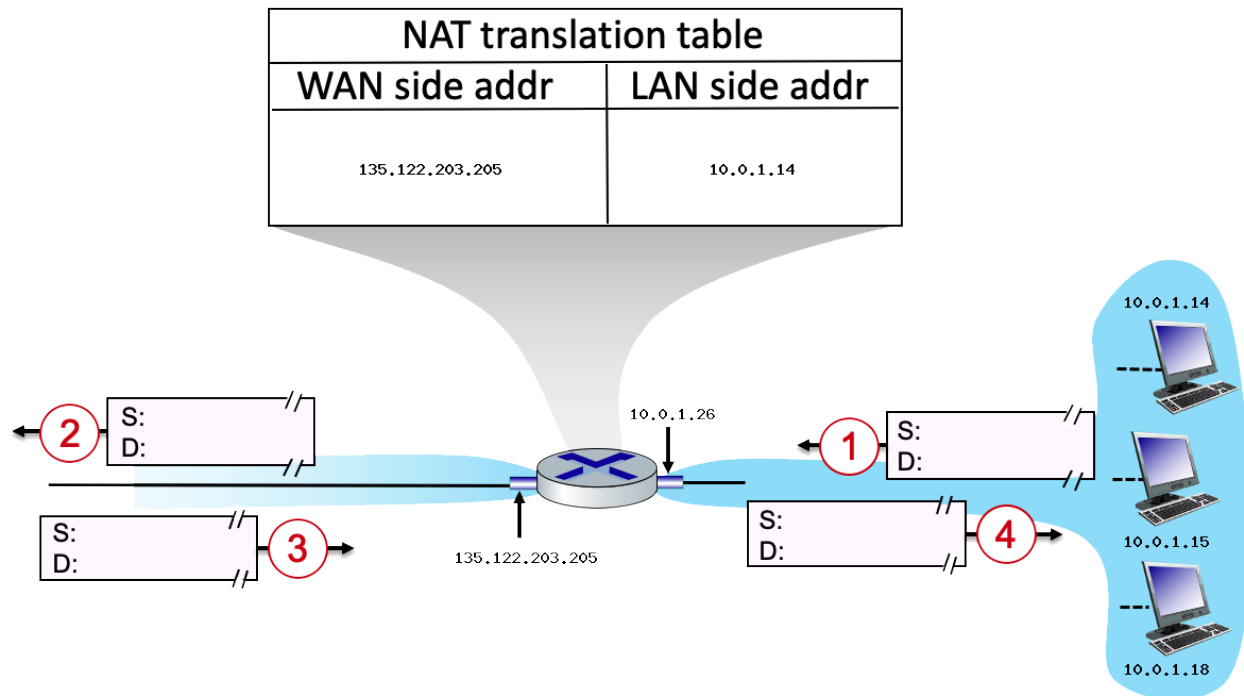
۱.۴. برای سوالات زیر توضیح مختصر بیاورید:

(آ) چرا روتر NAT باید آدرس مبدا را هنگام خروج بسته از شبکه محلی تغییر دهد؟

(ب) روتر NAT چگونه با استفاده از جدول ترجمه، بسته‌های برگشتی را به میزبان صحیح تحویل می‌دهد؟

۲.۴. برای چهار مرحله نشان داده شده در شکل، آدرس‌های مبدا و مقصد را مشخص کنید:

(آ) مرحله 1، بسته پس از ترک میزبان و قبل از رسیدن به روتر : آدرس‌های Source و Destination چیست؟



شکل ۲: سناریوی ترجمه آدرس در روتر NAT

(ب) مرحله ۲، پس از ارسال بسته توسط روتر به اینترنت : آدرس های Source و Destination چیست؟
آیا پورت مبدا تغییر می کند؟

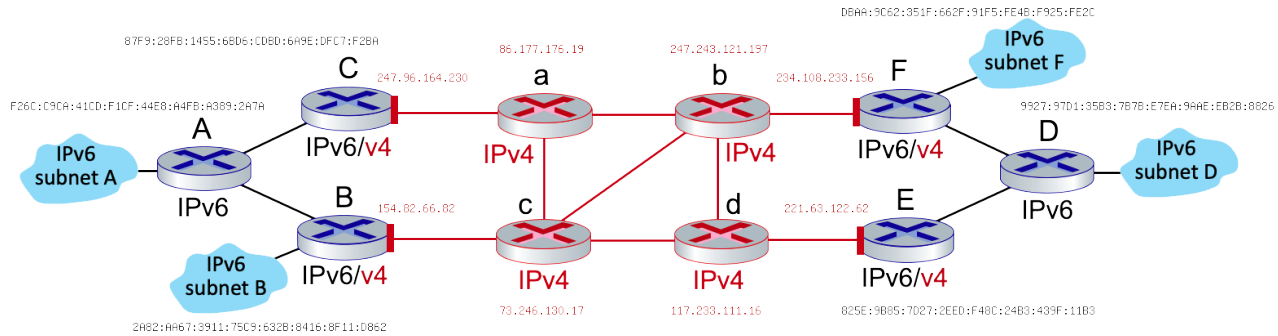
(ج) مرحله ۳، پس از بازگشت بسته از اینترنت و قبل از ورود به روتر : آدرس های Source و Destination چیست؟

(د) مرحله ۴، پس از ارسال بسته توسط روتر به میزبان داخلی : آدرس های Source و Destination چیست؟ آیا ورودی جدیدی در جدول NAT ساخته شده است؟

سوال ۵: تونل زنی و کپسوله سازی IPv6 روی شبکه IPv4

در شکل زیر چند زیرشبکه IPv6 توسط روترهای فقط-IPv6، فقط-IPv4 و روترهای دو قابلیتی به یکدیگر متصل شده اند. فرض کنید یک میزبان در زیرشبکه D می خواهد یک دیتاگرام IPv6 را به یک میزبان در زیرشبکه A ارسال کند. مسیر ارسال به صورت زیر است:

$$D \rightarrow F \rightarrow b \rightarrow a \rightarrow C \rightarrow A$$



شکل ۳: تونل زنی IPv6 روی زیرساخت IPv4

۱.۵. برای سوالات زیر توضیح مختصر بیاورید:

(آ) چرا در بخشی از مسیر، دیتاگرام IPv6 باید داخل یک دیتاگرام IPv4 قرار گیرد؟

(ب) ورودی تونل و خروجی تونل چه نقشی در مسیر ایفا می کنند؟

۲.۵ (آ) در ارسال از F به b: نوع دیتاگرام چیست؟ آدرس مبدا و مقصد چه هستند؟ آیا این دیتاگرام کپسوله شده است؟

(ب) در ارسال از b به a: نوع دیتاگرام چیست؟ آدرس مبدا و مقصد چه هستند؟ اگر کپسوله شده است، آدرس های مبدا و مقصد دیتاگرام درونی چیست؟

(ج) در ارسال از C به A: نوع دیتاگرام چیست؟ آدرس مبدا و مقصد چه هستند؟ آیا دیتاگرام کپسوله شده است؟

(د) ورودی تونل کدام روتر است؟ خروجی تونل کدام روتر است؟ کدام پروتکل درون دیگری قرار می گیرد؟

سوال ۶: جداول فلو در OpenFlow

در شکل زیر یک شبکه چهار روتره نشان داده شده است که در آن، مسیریابی بسته ها توسط جداول فلو (مثلاً با OpenFlow در یک کنترلر SDN) انجام می شود، نه توسط جدول مسیریابی سنتی. زیرشبکه های متصل به هر روتر به صورت زیر هستند:

□ روتر s1: شبکه 128.119/16

□ روتر s2: شبکه 128.120/16

□ روتر s3: شبکه 128.121/16

□ روتر s4: شبکه 128.122/16

هدف این است که رفتار زیر را پیاده سازی کنیم:

□ بسته های TCP که از شبکه متصل به s2 می آیند و به شبکه متصل به s3 می روند، باید از مسیر

$$s2 \rightarrow s1 \rightarrow s3$$

عبور کنند.

□ بسته های UDP با همین مبداء و مقصد باید مستقیماً از مسیر

$$s2 \rightarrow s3$$

عبور کنند.

فرض کنید هر روتر یک جدول match + action دارد و هر بسته ای که با هیچ قانونی در جدول مچ نشود حذف می شود.

۱.۶. برای سوالات زیر توضیح مختصر بیاورید

(آ) توضیح دهید منظور از یک قانون match + action در OpenFlow چیست و چرا در این سناریو باید قوانین تا حد ممکن دقیق تعریف شوند.

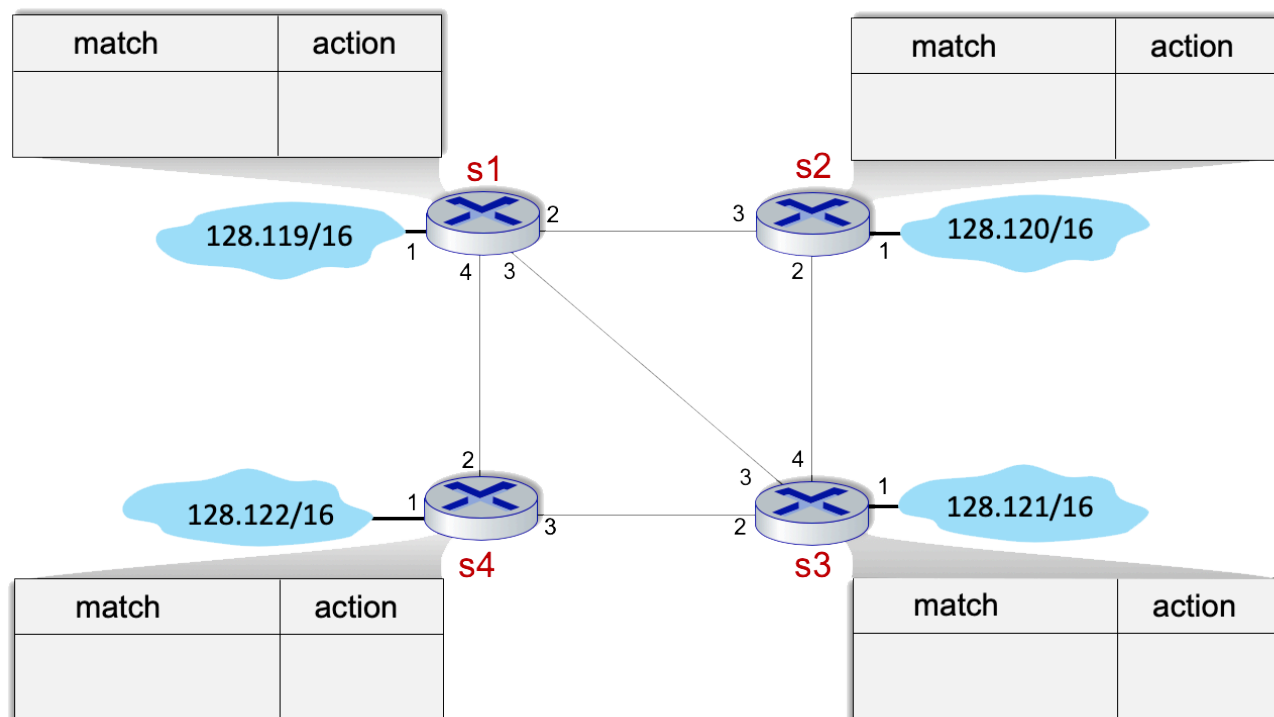
(ب) اگر بسته ای از شبکه متصل به s4 به مقصد شبکه متصل به s3 ارسال شود، با توجه به قوانین مورد نیاز در این سوال، چه اتفاقی برای آن بسته رخ می دهد؟ چرا؟

۲.۶. قوانین مورد نیاز را به صورت match + action برای روترهای زیر مشخص کنید تا رفتار گفته شده پیاده سازی شود. در هر قانون، فیلدهای IP Src, IP Dst, Src Port, Dst Port, IP Proto, و عمل Action (شامل شماره اینترفیس) را مشخص کنید.

(آ) جدول فلو روتر s1 (فقط برای بسته های TCP مربوطه).

(ب) دو قانون مورد نیاز در روتر s2: یکی برای بسته های TCP و یکی برای بسته های UDP با همان مبداء و مقصد.

(ج) دو قانون مورد نیاز در روتر s3 برای تحویل بسته های TCP و UDP به زیرشبکه محلی آن.



شکل ۴: شبکه چهار روتره با جداول فلو

سوال ۷: IP Fragmentation

می‌خواهیم یک دیتاگرام IP با اندازه 2400 بایت را روی لینکی بفرستیم که MTU آن 700 بایت است. فرض کنید شماره شناسه دیتاگرام اولیه 422 است. این دیتاگرام باید تکه تکه شود. مشخص کنید این دیتاگرام به چند تکه تقسیم می‌شود و مقادیر فیلدهای مرتبط با تکه سازی در هر تکه چیست.

سوال ۸: تاثیر NAT بر برنامه های P2P

فرض کنید نظیری به نام آلیس بعد از پرس و جو متوجه می‌شود که نظیری به نام باب فایل مورد نظر او را دارد. فرض کنید آلیس و باب هر دو پشت NAT قرار دارند. روشی طراحی کنید که آلیس و باب بدون نیاز به پیکربندی خاص برای NAT بتوانند یک اتصال TCP با یکدیگر برقرار کنند. اگر در طراحی چنین روشی مشکل دارید، دلیل آن را توضیح دهید.

سوال ۹: مدار مجازی و شماره های VC

یک شبکه مدار مجازی را در نظر بگیرید که از یک فیلد شماره VC دو بیتی استفاده می کند. فرض کنید این شبکه می خواهد یک مدار مجازی روی چهار لینک A، B، C و D ایجاد کند. در حال حاضر، هر یک از این لینک ها در حال عبور دادن دو مدار مجازی هستند که شماره VC آنها را در جدول زیر می بینید:

لینک A	لینک B	لینک C	لینک D
00	01	10	11
01	10	11	00

در هنگام پاسخ دادن به پرسش های زیر در نظر داشته باشید که هر یک از مدارهای مجازی موجود فقط از یکی از این چهار لینک عبور می کنند.

الف. اگر همه لینک های در امتداد یک مسیر بایستی شماره VC یکسان داشته باشند، چه شماره ای را می توان به این مدار مجازی جدید اختصاص داد؟

ب. اگر لینک های در امتداد یک مسیر بتوانند شماره های VC متفاوت داشته باشند (به عبارت دیگر، جدول های هدایت شماره های VC را به یکدیگر تبدیل کنند)، از چه تعداد ترکیب چهار شماره ای (یک شماره برای هر لینک) می توان استفاده کرد؟

سوال ۱۰: معماری سوییچ داخلی در مسیریاب

فرض کنید دو بسته مختلف دقیقاً در یک لحظه به دو پورت ورودی یک مسیریاب می رسند. همچنین فرض می کنیم هیچ بسته دیگری در هیچ جای این مسیریاب وجود ندارد.

الف. فرض کنید این دو بسته باید به دو پورت خروجی متفاوت هدایت شوند. اگر سوییچ داخلی این مسیریاب از نوع گذرگاه مشترک باشد، آیا این دو بسته می توانند به طور همزمان از سازه سوییچینگ عبور کنند؟

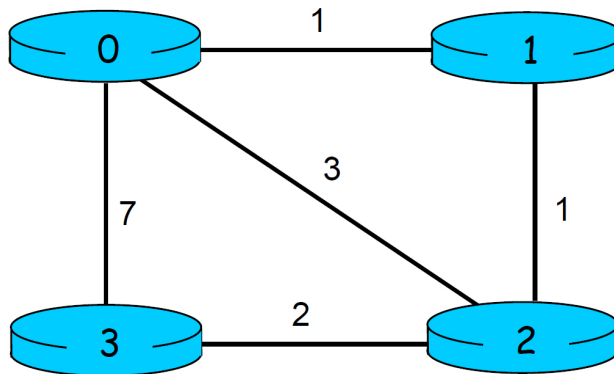
ب. باز هم فرض کنید این دو بسته باید به دو پورت خروجی متفاوت هدایت شوند. اگر سوییچ داخلی این مسیریاب از نوع نوار متقاطع باشد، آیا این دو بسته می توانند به طور همزمان از سازه سوییچینگ عبور کنند؟

پ. اکنون فرض کنید این دو بسته باید هر دو به یک پورت خروجی هدایت شوند. اگر سوییچ داخلی این مسیریاب از نوع نوار متقاطع باشد، آیا این دو بسته می توانند به طور همزمان از سازه سوییچینگ عبور کنند؟

سوال ۱۱

مرور کلی

در این تکلیف برنامه‌نویسی، شما مجموعه‌ای از پروسه‌های «توزیع‌شده» را خواهید نوشت که یک مسیریابی بردار فاصله (distance vector) ناهمگام توزیع‌شده را برای شبکه نشان داده شده در زیر پیاده‌سازی می‌کنند.



تکلیف اصلی

پروسه‌هایی که خواهید نوشت

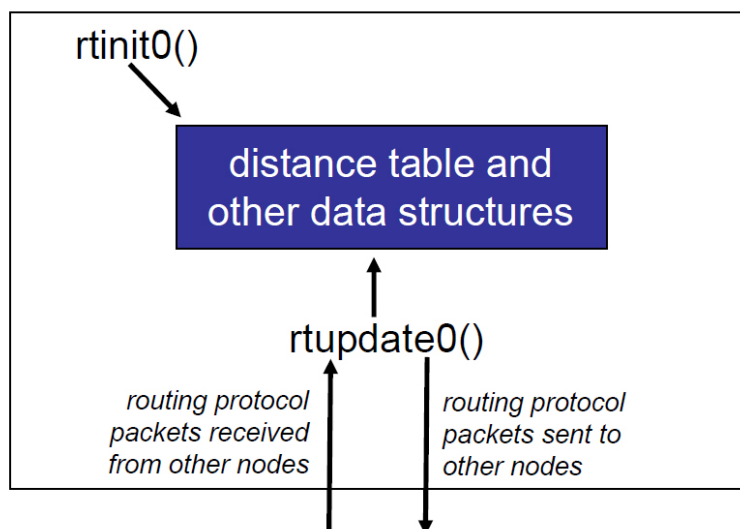
برای بخش اصلی تکلیف، شما باید پروسه‌های زیر را بنویسید که به صورت ناهمگام در محیط شبیه‌سازی شده‌ای که ما برای این تکلیف نوشته‌ایم، «اجرا» خواهند شد.
برای گره 0، شما پروسه‌های زیر را خواهید نوشت:

□ `rtinit0()` این پروسه یک بار در ابتدای شبیه‌سازی فراخوانی خواهد شد. `rtinit0()` هیچ آرگومانی ندارد. این پروسه باید جدول فاصله (distance table) (شکل زیر را ببینید) در گره 0 را مقداردهی اولیه کند تا هزینه‌های مستقیم 1، 3 و 7 به ترتیب به گره‌های 1، 2 و 3 را منعکس کند. در شکل بالا، تمام پیوندها دوطرفه هستند و هزینه‌ها در هر دو جهت یکسان می‌باشند. پس از مقداردهی اولیه جدول فاصله و هر ساختار داده دیگری که مورد نیاز پروسه‌های گره 0 شما باشد، باید هزینه‌های مسیرهای با کمترین هزینه خود، یعنی بردار فاصله (distance vector) خود را، به تمام گره‌های دیگر شبکه، برای همسایگان مستقیماً متصل خود (در این مورد، 1، 2 و 3) ارسال کند. این اطلاعات کمترین هزینه در یک بسته مسیریابی (routing packet) با فراخوانی پروسه `tolayer2()`، همانطور که در زیر توضیح داده شده، به گره‌های همسایه ارسال می‌شود. قالب بسته مسیریابی نیز در زیر شرح داده شده است.

□ `rtupdate0(struct rtpkt *rcvdpkt)` این پروسه زمانی فراخوانی می‌شود که گره 0 یک بسته مسیریابی دریافت کند که توسط یکی از همسایگان مستقیماً متصل به آن ارسال شده است. پارامتر `*rcvdpkt` یک اشاره‌گر به بسته دریافت شده است. `rtupdate0()` «قلب» الگوریتم بردار فاصله است. مقداری که در یک بسته مسیریابی از گره دیگری مانند `j` دریافت می‌کند، حاوی هزینه‌های کوتاه‌ترین مسیر فعلی `j` (بردار فاصله `j`) به تمام گره‌های دیگر شبکه است. `rtupdate0()` از این مقادیر دریافت شده برای به‌روزرسانی «جدول فاصله» خود استفاده می‌کند. جدول فاصله درون هر گره، ساختار داده اصلی مورد استفاده توسط الگوریتم بردار فاصله است. برای شما راحت بود که جدول فاصله را به عنوان یک آرایه `4` در `4` از `int` تعریف کنید، که در آن ورودی `[i,j]` در جدول فاصله در گره `0`، هزینه محاسبه شده فعلی گره `0` به گره `i` از طریق همسایه مستقیم `j` است؛ گره `0` ورودی‌های جدول فاصله خود را برای ستون `j` پس از دریافت بردار فاصله `j` (دوباره) محاسبه می‌کند. اگر `0` مستقیماً به `j` متصل نباشد، می‌توانید این ورودی را نادیده بگیرید. ما از این قرارداد استفاده خواهیم کرد که مقدار صحیح `999` «بی‌نهایت» است.

همانطور که توسط الگوریتم بردار فاصله که در کلاس و در متن درس مطالعه کردیم مشخص شده است، اگر کمترین هزینه خود گره `0` به گره دیگری در نتیجه این به‌روزرسانی تغییر کند، گره `0` همسایگان مستقیماً متصل خود را از این تغییر در کمترین هزینه با ارسال یک بسته مسیریابی به آنها مطلع می‌سازد. به یاد بیاورید که در الگوریتم بردار فاصله، فقط گره‌های مستقیماً متصل بسته‌های مسیریابی را مبادله می‌کنند. بنابراین گره‌های `1` و `2` با یکدیگر ارتباط برقرار می‌کنند، اما گره‌های `1` و `3` با یکدیگر ارتباط برقرار نخواهند کرد.

پروسه‌های مشابهی برای گره‌های `1`، `2` و `3` تعریف شده‌اند. بنابراین، شما در مجموع `8` پروسه خواهید نوشت: `rtinit0()`، `rtinit1()`، `rtinit2()`، `rtinit3()`، `rtupdate0()`، `rtupdate1()`، `rtupdate2()`، `rtupdate3()`



واسط‌های نرم‌افزاری

پروسه‌های توصیف شده در بالا آنهایی هستند که شما خواهید نوشت. ما پروسه‌های زیر را نوشته‌ایم که می‌توانند توسط پروسه‌های شما فراخوانی شوند:

□ `tolayer2(struct rtpkt pkt2send)` که در آن `rtpkt` ساختار زیر است، که قبلاً برای شما تعریف شده است. پروسه `tolayer2()` در فایل `distance_vector.c` تعریف شده است.

```
extern struct rtpkt {
    int sourceid; /* id of node sending this pkt, 0, 1, 2, or 3 */
    int destid; /* id of router to which pkt being sent
                (must be an immediate neighbor) */
    int mincost[4]; /* min cost to node 0 ... 3 */
};
```

توجه داشته باشید که به `tolayer2()` یک ساختار پاس داده می‌شود، نه یک اشاره‌گر به ساختار.

□ `printdt0()` جدول فاصله را برای گره 0 به صورت زیبا چاپ (pretty print) می‌کند. به آن یک اشاره‌گر به ساختاری از نوع `distance_table` پاس داده می‌شود. `printdt0()` و اعلان ساختار برای جدول فاصله گره 0 در فایل `node0.c` تعریف شده‌اند. پروسه‌های مشابه چاپ زیبا برای شما در فایل‌های `node1.c`، `node2.c` و `node3.c` تعریف شده‌اند.

محیط شبکه شبیه‌سازی شده

پروسه‌های شما `rtinit0()`، `rtinit1()`، `rtinit2()`، `rtinit3()` و `rtupdate0()`، `rtupdate1()`، `rtupdate2()`، `rtupdate3()` بسته‌های مسیریابی (که قالب آنها در بالا توضیح داده شد) را به محیط ارسال می‌کنند. محیط، بسته‌ها را به ترتیب و بدون از دست دادن به مقصد مشخص شده تحویل می‌دهد. فقط گره‌های مستقیماً متصل می‌توانند ارتباط برقرار کنند. تأخیر بین فرستنده و گیرنده متغیر (و ناشناخته) است. وقتی شما پروسه‌های خود و پروسه‌های من را با هم کامپایل می‌کنید و برنامه حاصل را اجرا می‌کنید، از شما خواسته می‌شود که فقط یک مقدار را در مورد محیط شبکه شبیه‌سازی شده مشخص کنید:

□ `Tracing`. تنظیم مقدار `tracing` روی 1 یا 2 اطلاعات مفیدی در مورد آنچه در داخل شبیه‌سازی در حال رخ دادن است (مثلاً، چه اتفاقی برای بسته‌ها و تایمرها می‌افتد) چاپ می‌کند. مقدار `tracing` برابر 0 این قابلیت را

خاموش می‌کند. مقدار tracing بزرگتر از 2 انواع پیام‌های عجیب و غریبی را نمایش می‌دهد که برای اهداف اشکال‌زدایی شبیه‌ساز خود من هستند.

یک مقدار tracing برابر 2 ممکن است برای شما در اشکال‌زدایی کدتان مفید باشد. باید به خاطر داشته باشید که پیاده‌سازان واقعی شبکه‌های زیربنایی ندارند که چنین اطلاعات خوبی در مورد آنچه قرار است برای بسته‌هایشان اتفاق بیفتد، ارائه دهند!

تکلیف اصلی

شما باید پرونده‌های `rtinit0()`، `rtinit1()`، `rtinit2()`، `rtinit3()` و `rtupdate0()`، `rtupdate1()`، `rtupdate2()`، `rtupdate3()` را بنویسید که با هم یک محاسبه توزیع‌شده و ناهمگام از جداول فاصله را برای توپولوژی و هزینه‌های نشان داده شده در شکل اول پیاده‌سازی کنند.

شما باید پرونده‌های خود را برای گره‌های 0 تا 3 در فایل‌هایی به نام `node0.c`، ...، `node3.c` قرار دهید. شما مجاز به تعریف هیچ متغیر سراسری (global variables) که خارج از یک فایل C معین قابل مشاهده باشد، نیستید (به عنوان مثال، هر متغیر سراسری که در `node0.c` تعریف می‌کنید، فقط باید در داخل `node0.c` قابل دسترسی باشد). این به منظور وادار کردن شما به رعایت قراردادهای کدنویسی است که اگر واقعاً پرونده‌ها را در چهار گره مجزا اجرا می‌کردید، مجبور به اتخاذ آن‌ها بودید. برای کامپایل پرونده‌های خود: `cc distance_vector.c node0.c node1.c node2.c node3.c`. شما می‌توانید یک کپی از فایل `distance_vector.c` را از پوشه `starter_code` در صفحه ایلرن درس بردارید. شما می‌توانید پرونده‌های `node*.c` و `distance_vector.c` را که به طور خاص برای linux اصلاح شده‌اند، از همان پوشه بردارید.

این تکلیف می‌تواند بر روی هر ماشینی که از C، C++، یا JAVA پشتیبانی می‌کند، تکمیل شود. این تکلیف از هیچ ویژگی UNIX استفاده نمی‌کند.

برای خروجی نمونه شما، پرونده‌های شما باید هر زمان که پرونده‌های `rtinit0()`، `rtinit1()`، `rtinit2()`، `rtinit3()` یا `rtupdate0()`، `rtupdate1()`، `rtupdate2()`، `rtupdate3()` شما فراخوانی می‌شوند، پیامی را چاپ کنند که زمان (از طریق متغیر سراسری من `clocktime` در دسترس است) را نشان دهد. برای `rtupdate0()`، `rtupdate1()`، `rtupdate2()`، `rtupdate3()` شما باید هویت فرستنده بسته مسیریابی که به پرونده شما پاس داده می‌شود، اینکه آیا جدول فاصله به‌روزرسانی شده است یا نه، محتویات جدول فاصله، و توصیفی از هر پیامی که در نتیجه هرگونه به‌روزرسانی جدول فاصله به گره‌های همسایه ارسال می‌شود را چاپ کنید.

خروجی نمونه باید یک لیست خروجی با مقدار TRACE برابر 2 باشد. جدول فاصله نهایی تولید شده در هر گره را هایلایت کنید. برنامه شما تا زمانی که هیچ بسته مسیریابی دیگری در حال انتقال در شبکه نباشد، اجرا خواهد شد،

در آن نقطه شبیه‌ساز ما خاتمه می‌یابد.

توجه داشته باشید که یک سند طراحی کوتاه مورد نیاز است. شما باید هر فرآیند خود را طوری برنامه‌ریزی کنید که هرگاه اقدامی انجام می‌دهد (مثلاً، ارسال یا دریافت پیام، تشخیص پایان ورودی، و غیره)، یک عبارت آموزنده چاپ کند تا بتوانید ببینید که فرآیندهای شما به درستی کار می‌کنند (یا نه!). این همچنین به TA اجازه می‌دهد تا از این خروجی تشخیص دهد که آیا فرآیندهای شما به درستی کار می‌کنند یا خیر. شما باید اسکرین‌شات‌ها (یا محتوای فایل، اگر فرآیند شما در حال نوشتن در یک فایل است) از این پیام‌های آموزنده را تحویل دهید.

امتیاز اضافی

شما باید دو پروسه، `rtlinkhandler0(int linkid, int newcost)` و `rtlinkhandler1(int linkid, int newcost)` را بنویسید، که اگر (و زمانی که) هزینه پیوند بین 0 و 1 تغییر کند، فراخوانی خواهند شد. این پروسه‌ها باید به ترتیب در فایل‌های `node0.c` و `node1.c` تعریف شوند. به این پروسه‌ها نام (id) گره همسایه در طرف دیگر پیوندی که هزینه‌اش تغییر کرده، و هزینه جدید پیوند پاس داده خواهد شد. توجه داشته باشید که وقتی هزینه یک پیوند تغییر می‌کند، این پروسه‌ها باید جدول فاصله را به‌روزرسانی کنند و ممکن است (یا نباشد) مجبور شوند بسته‌های مسیریابی به‌روزرسانی شده را به گره‌های همسایه ارسال کنند.

برای تکمیل بخش پیشرفته تکلیف، شما باید مقدار ثابت `LINKCHANGES` (خط 3 در `distance_vector.c`) را به 1 تغییر دهید. جهت اطلاع، هزینه پیوند از 1 به 20 در زمان 10000 تغییر خواهد کرد و سپس در زمان 20000 به 1 باز خواهد گشت. پروسه‌های شما در این زمان‌ها فراخوانی خواهند شد.

ما مجدداً اکیداً توصیه می‌کنیم که ابتدا تکلیف اصلی را پیاده‌سازی کنید و سپس کد خود را برای پیاده‌سازی تکلیف امتیاز اضافی گسترش دهید. این کار اتلاف وقت نخواهد بود.

نسخه JAVA این تکلیف

مستندات بالا پروژه را با جزئیات توصیف می‌کند. در اینجا ما پیوندی به کد مورد نیاز برای انجام تکلیف در JAVA ارائه می‌دهیم. مطمئن شوید که مطالب بالا را درک کرده‌اید.

کد JAVA که نیاز دارید را می‌توانید در اینجا پیدا کنید. در اینجا فایل‌های جداگانه‌ای که نیاز خواهید داشت، آمده‌اند:

`Entity.java`, `Entity0.java`, `Entity1.java`, `Entity2.java`, `Entity3.java`, `NetworkSimulator.java`, `Event.java`, `Packet.java`, `EventList.java`, `EventListImpl.java`, `Project.java`

شما سازنده‌های (`Entity0.java`, `Entity1.java`, `Entity2.java`, `Entity3.java`) را خواهید نوشت که مشابه `rtinit0()`, `rtinit1()`, `rtinit2()` و `rtinit3()` در نسخه C هستند. شما همچنین باید متدهای

update() را برای Entity0.java، Entity1.java، Entity2.java، و Entity3.java بنویسید که مشابه rtupdate0()، rtupdate1()، riupdate2() و rtupdate3() در نسخه C هستند. توجه داشته باشید که کد Java به شما اجازه می‌دهد با ارسال بسته‌های نادرست از طریق متد toLayer2() از NetworkSimulator، خود را به دردسر بیندازید. پس لطفاً در آنجا بسیار مراقب باشید.

نکات تکمیلی

۱. در پوشه starter_code می‌توانید فایل‌های مورد نیازتان را پیدا کنید.
۲. **حتماً اگر بخش امتیازی را انجام می‌دهید، با کامنت گذاری این بخش را مشخص کنید و در گزارش توضیح دهید.**
۳. گزارش مجزا برای سوال آخر تمرین الزامی است!

نکاتی که باید توجه داشته باشید:

- الف) مهلت ارسال در سربرگ تمرین همچنین در ایلرن درج شده است.
- ب) کلیه تمرینات (غیر از تمرینات سرکلاسی) از طریق ایلرن دریافت می شوند و دیگر شیوه های ارسال تمرین پذیرفته نیست.
- ج) در صورتی که تمرینات در قالب $\text{\LaTeX}$ و تنها در Template تمرینات ارسال شوند، ۵٪ نمره‌ی اضافی به عنوان امتیاز در نظر گرفته خواهد شد. (Template در ایلرن در دسترس است).
- د) فایل تمرینات ارسالی باید شامل فایل‌های مورد نیاز باشد. در صورتی که تمرینات در قالب $\text{\LaTeX}$ تهیه شده باشند، لازم است فایل‌های مورد نیاز جهت اجرای فایل $\text{\LaTeX}$ به همراه فایل PDF نیز ارسال شوند. نام فایل را به صورت زیر انتخاب کنید:

CN#_StudentNumber#_StudentName

- ه) ارسال با تأخیر تمرین‌ها و پروژه‌های درس صرفاً بر اساس قوانین ذکرشده در فصل صفر امکان‌پذیر خواهد بود.